Systemsoftware Übung 1

Philipp Heigl 1767707

Aufgabe 1

Programm: Ein Programm ist eine Folge von Anweisungen, die als ausführbare Datei auf einer Festplatte liegen. Es beschreibt, welche Operationen ausgeführt werden sollen, führt diese jedoch nicht selbstständig aus. Erst wenn ein Programm gestartet wird, erzeugt das Betriebssystem einen Prozess, in dem die Anweisungen ausgeführt werden.

Prozess: Ein prozess ist eine aktive, laufende Instanz eines Programms. Es wird vom Betriebssystem verwaltet und besitzt einen eigenen virtuellen Adressraum sowie Ressourcen wie Speicher und geöffnete Dateien. Jeder Prozess enthält mindestens einen Thread, der die Anweisungen des Programms ausführt. Im Gegensatz zu Threads desselben Prozesses sind verschiedene Prozesse grundsätzlich voneinander isoliert und besitzen getrennte Adressräume.

Thread: Ein Thread ist eine Ausführeinheit innerhalb eines Prozesses. Ein Prozess hat mindestens einen Thread, kann jedoch mehrere Threads enthalten, die unterschiedliche Aufgaben gleichzeitig oder nebenläufig ausführen. Dabei teilen sich die Threads eines Prozesses denselben Adressraum und können auf gemeinsame Ressourcen zugreifen. Im Gegensatz zu einem Prozess benötigt ein Thread keinen eigenen Adressraum, wodurch die Zusammenarbeit und der Datenaustausch zwischen Threads einfacher und effizienter ist.

Zusammenhang: Ein Programm enthält die auszuführenden Anweisungen. Wird es gestartet, entsteht ein Prozess, der die notwendigen Ressourcen bereitstellt. Innerhalb dieses Prozesses führen ein oder mehrere Threads die Anweisungen aus. Während Prozesse voneinander isoliert sind, können Threads desselben Prozesses direkt auf gemeinsame Daten zugreifen und dadurch effizient zusammenarbeiten.

Aufgabe 2

a) Die Beschleunigung eines Prozesses durch zusätzliche Prozessorkerne und eine höhere Taktfrequenz ist aus verschiedenen Gründen begrenzt.

Zusätzliche Prozessorkerne: Nicht alle Teile eines Programms können parallel ausgeführt werden. Bestimmte Berechnungen müssen nacheinander erfolgen, da sie voneinander abhängig sind. Dieser serielle Anteil kann durch zusätzliche Kerne nicht beschleunigt werden. Nach dem Amdahlschen Gesetz begrenzt dieser Anteil die maximal mögliche Beschleunigung. Außerdem entsteht bei der Parallelisierung zusätzlicher Aufwand durch die Koordination und Synchronisation der Threads.

Höhere Taktfrequenz: Eine höhere Taktfrequenz ermöglicht es einem Prozessorkern mehr Taktzyklen pro Sekunde auszuführen. Allerdings steigen dabei der Energieverbrauch und die Wärmeentwicklung, wodurch physikalische und technische Grenzen entstehen. Zudem hängt die Ausführungsgeschwindigkeit nicht ausschließlich von der Taktfrequenz ab, sondern beispielsweise auch von Speicherzugriffen und der Prozessorarchitektur.

Fazit: Die Beschleunigung eines Prozesses ist sowohl durch den nicht parallelisierbaren Programmanteil als auch durch technische Grenzen der Hardware begrenzt. Daher können weder zusätzliche Kerne noch eine höhere Taktfrequenz einen Prozess unbegrenzt beschleunigen.

b) Formel: $T_{Multicore}(s,n) = T_{Singlecore}(s) + T_{Singlecore}\left(\frac{1-s}{n}\right)$ Basis ist ein Prozess, der auf einem Kern 1s dauert.

Parallelisierbarer Anteil: 25% Kerne: 2

$0,25 + \frac{1-0,25}{2} = 0,875$

Speedup: $\frac{1}{0,875} \approx 1,14$ => Das Programm wird somit theoretisch um den Faktor 1,14 beschleunigt.

Parallelisierbarer Anteil	2 Kerne	4 Kerne	8 Kerne	16 Kerne
25%	1,14	1,23	1,28	1,31
50%	1,33	1,60	1,78	1,88
75%	1,60	2,29	2,91	3,37

Aufgabe 3

Zur Untersuchung der Parallelisierung wurde das Programm mit 1,2,4,8,16,32 Threads ausgeführt. Dabei wurden .txt-Dateien auf dem Desktop mit insgesamt 323.065 Zeilen verarbeitet.

Folgend Laufzeiten wurden gemessen:

Anzahl Threads	Benötigte Zeit in ms
1	117,05
2	95,63
4	81,89
8	76,05
16	82,44
32	84,65

Die Ergebnisse zeigen, dass die Ausführungszeit zunächst mit steigender Thread-Anzahl abnimmt. Die kürzeste Laufzeit wurde mit 8 Threads und 76,05 ms erreicht. Gegenüber der Ausführung mit einem Thread (117,05 ms) entspricht das einer Beschleunigung um den Faktor 1,54.
Ab 16 Threads steigt die Ausführungszeit wieder an. Dies lässt sich möglicherweise durch den zusätzlichen Verwaltungsaufwand der Threads und die begrenzten Hardware-Ressourcen erklären. Da mehrere Threads gleichzeitig auf Dateien zugreifen, kann außerdem die Geschwindigkeit des Dateisystems zum begrenzenden Faktor werden.
Die Messungen zeigen, dass eine höhere Thread-Anzahl nicht zwangsläufig zu einer kürzeren Ausführungszeit führt.